

**UNIVERSIDADE DE PERNAMBUCO
ESCOLA POLITÉCNICA DE PERNAMBUCO
GRADUAÇÃO EM ENGENHARIA DA COMPUTAÇÃO**

Projeto Calango-Tech

Robô Explorador com Otimização Energética por Aprendizado por Reforço

Grupo:

José Mário da Silva Filho
Pedro Henrique França Dias
Victor José de Carvalho Ferreira
Vinicius Moura de Oliveira

**RECIFE
2026**

Sumário

1. Visão Geral do Projeto	3
2. Desenvolvimento em Simulação (CoppeliaSim)	3
2.1. Ambiente e Stack Técnica	3
2.2. Modelagem do Aprendizado por Reforço	3
2.3. Action Shaping e Métrica de Energia	4
2.4. Detecção de Capotamento por Matriz de Rotação	4
2.5. Hiperparâmetros do Treinamento	4
2.6. Arquitetura de Software da Simulação	5
3. Resultados da Simulação	5
3.1. Robô Esteira (Caterpillar)	5
3.2. Robô de Seis Rodas: Construção e Validação	6
3.3. Otimização de Tração (Slip Ratio)	6
3.4. Comparativo Esteira × Seis Rodas	7
4. Protótipo Físico: Arquitetura de Hardware	7
4.1. Visão Geral e Custo do Protótipo	7
4.2. Pinagem e Conexões	8
4.3. Cronograma de Desenvolvimento e Validação	8
5. Firmware Embarcado (ESP32)	9
5.1. Controle Tank Drive via Xbox One S	9
5.2. Leitura da IMU e Filtro Complementar	9
5.3. Medição de Umidade com Servo	10
5.4. Telemetria: Monitor Serial e Dashboard Web	10
5.5. Versões do Firmware	10
6. Resultados do Protótipo Físico	11
6.1. Validação dos Sensores	11
6.2. Testes de Mobilidade em Rampa	11
6.3. Limitações Identificadas	12
7. Comparativo Simulação × Realidade (Gap Sim-to-Real)	12
8. Guia Passo a Passo de Execução	13
8.1. Pré-requisitos	13
8.2. Executando a Simulação	13
8.3. Gravando e Operando o Protótipo Físico	13
9. Conclusões e Trabalhos Futuros	14

1. Visão Geral do Projeto

O Projeto Calango-Tech consiste no desenvolvimento do robô Calango Tech, um explorador concebido para vencer um terreno acidentado com rampas de inclinações distintas e realizar a medição de uma grandeza ambiental (umidade do solo) em cada patamar, otimizando a eficiência energética em relação a um controle de aceleração fixa (baseline), sem capotar nem perder tração.

O trabalho consolidou-se em duas frentes complementares, com escopos distintos e claramente delimitados:

- Simulação (CoppeliaSim + Aprendizado por Reforço): treinamento de uma política de controle por Reinforcement Learning (PPO) que decide, a cada instante, a aceleração dos motores. Foram modeladas e comparadas duas arquiteturas mecânicas — esteira (caterpillar) e seis rodas — com análise de otimização de tração e eficiência energética.
- Protótipo físico (ESP32 + esteira, teleoperado): construção de um veículo de esteira real, com tração diferencial (tank drive) comandada por controle Xbox One S via Bluetooth, instrumentado com IMU, sensor de umidade e servo de medição, além de telemetria por monitor serial e dashboard web.

Delimitação de escopo: por definição de projeto, o protótipo físico foi concebido desde o início para se locomover por controle Bluetooth — a locomoção autônoma não integrou o escopo da frente física. A política de Aprendizado por Reforço é, portanto, resultado da frente de simulação, ao passo que o protótipo teve por finalidade validar a mecânica de esteira, a percepção (sensores) e a mobilidade em rampa sob teleoperação.

O código-fonte do firmware embarcado está publicado em: <https://github.com/MF853/Calango-Tech>

2. Desenvolvimento em Simulação (CoppeliaSim)

A frente de simulação foi construída sobre o CoppeliaSim 4.10, acoplado a um ambiente de aprendizado por reforço em Python. A comunicação entre simulador e agente é feita pela ZMQ Remote API em modo stepping, garantindo determinismo entre a decisão do agente e a evolução da física.

2.1. Ambiente e Stack Técnica

- Simulador: CoppeliaSim 4.10, motor de física Bullet 2.78, passo $dt = 50$ ms.
- Aprendizado por Reforço: Stable-Baselines3 (PPO, política MlpPolicy) sobre ambiente Gymnasium customizado (CrawlerEnv). Monitoramento por TensorBoard.
- Robô esteira (caterpillar): 8 motores dinâmicos (4 por lado), torque $1000 \text{ N}\cdot\text{m}$, coeficiente de atrito $\mu = 2,0$ e raio de roda $0,0704 \text{ m}$.
- Robô de seis rodas: 6 motores dinâmicos (3 por lado), torque $1000 \text{ N}\cdot\text{m}$ e raio de roda $0,0650 \text{ m}$.

2.2. Modelagem do Aprendizado por Reforço

O agente observa um vetor de 7 dimensões e produz uma única ação contínua no intervalo $[0, 1]$, convertida no throttle aplicado aos motores:

```
[ componente_de_subida, tilt_normalizado, altura_relativa,
  ganho_de_altura, velocidade_normalizada, acao_anterior,
  velocidade_para_frente ]
```

A função de recompensa premia o avanço e o ganho de altura e penaliza capotamento, patinagem e consumo de energia, de modo que a política ótima cumpra a missão gastando o mínimo de energia sem tombar.

2.3. Action Shaping e Métrica de Energia

O avanço metodológico decisivo foi o action shaping: restringiu-se a ação à faixa fisicamente útil de throttle por meio da transformação linear

```
throttle = 0.6 + acao * 0.4      # faixa util [0.6, 1.0]
```

Essa restrição reduziu a convergência do treinamento de aproximadamente 10 iterações para apenas 2, mantendo a estabilidade da política.

A energia é estimada por um proxy físico proporcional ao quadrado da velocidade alvo:

```
E = Σ ( 0.005 * velocidade_alvo² )
```

A base é a potência $P = \tau \cdot \omega$; como em regime de subida o torque cresce com a velocidade ($\tau \propto \omega$), tem-se $P \propto \omega^2$. A constante 0,005 corresponde a $k \cdot dt$, com $k \approx 0,1 \text{ W}/(\text{rad/s})^2$ e $dt = 0,05 \text{ s}$. Daí decorre que andar mais devagar economiza energia de forma quadrática — princípio que a política aprende de forma autônoma.

2.4. Detecção de Capotamento por Matriz de Rotação

Para medir a inclinação de forma robusta, evitou-se o uso de ângulos de Euler (ambíguos sob guinada). Utiliza-se o elemento `mat[10]` da matriz de rotação do robô no referencial do mundo — o cosseno do ângulo entre o eixo Z do robô e o Z do mundo. A medida é invariante à guinada, não confundindo uma curva com um tombamento. Considera-se capotamento acima de 60° .

2.5. Hiperparâmetros do Treinamento

Os hiperparâmetros do PPO foram ajustados para privilegiar a estabilidade do aprendizado em um terreno multi-rampa. Reduziu-se a taxa de aprendizado e o número de épocas em relação aos valores padrão — atualizações menores diminuem o risco de colapso da política enquanto a função de valor ainda não convergiu — e elevou-se o coeficiente de entropia, ampliando a exploração exigida por um terreno com três rampas distintas. A configuração é idêntica para as duas arquiteturas, garantindo um comparativo justo:

```
model = PPO(
    "MlpPolicy", env,
    learning_rate = 1.5e-4,    # atualizacoes menores = mais estabilidade
    n_steps       = 2048,
    batch_size    = 64,
    n_epochs      = 5,         # menos passes = menos overfitting a ruído
    gamma         = 0.99,
    gae_lambda    = 0.95,
```

```
ent_coef      = 0.015,      # exploracao ampliada p/ terreno multi-rampa
)
```

O treinamento é limitado a um teto de 200.000 timesteps, mas encerra-se antes por early-stop assim que as metas de recompensa e taxa de sucesso são atingidas.

2.6. Arquitetura de Software da Simulação

A frente de simulação organiza-se em scripts especializados, cada um com uma responsabilidade única — treino, execução da missão, medição ou diagnóstico:

Arquivo	Função
treinar_robô.py	Treino PPO do robô esteira (ambiente CrawlerEnv, reward shaping e callbacks)
testar_robô.py	Executa a missão da esteira; modo baseline pela flag --constante
treinar_robô_6rodas.py	Treino PPO do robô de seis rodas (mesma configuração, para comparação justa)
testar_robô_6rodas.py	Executa a missão do robô de seis rodas; também com modo baseline
teste_6rodas_gonogo.py	Teste de viabilidade física (GO/NO-GO) aplicado antes de treinar
medir_tracao.py	Varredura de throttle e medição do slip ratio
diagnostico_sim.py	Verificação de sanidade da cena, da física e dos sensores

3. Resultados da Simulação

3.1. Robô Esteira (Caterpillar)

O robô de esteira é a arquitetura de referência do projeto — e, como se verá na Seção 4, é também a arquitetura materializada no protótipo físico. Todos os indicadores-alvo (KPIs) foram atingidos:

KPI	Antes	Depois (obtido)
Capotamento (inclinação > 60°)	65%	0% (meta < 5%)
Pitch máximo na subida	50°	≈ 28°
Taxa de sucesso da missão	10%	100% (meta ≈ 95%)
ep_rew_mean (recompensa média)	-20	+35 a +50
Economia de energia vs baseline 0,8	—	≈ 30%

Além do sucesso integral e do capotamento nulo, a esteira demonstrou dominância de Pareto no espaço energia × tempo: a política treinada domina o baseline manual em consumo sem sacrifício relevante de tempo.

3.2. Robô de Seis Rodas: Construção e Validação

Para o estudo comparativo, modelou-se uma segunda arquitetura de seis rodas com tração diferencial. Antes de investir horas de treinamento, aplicou-se um teste de viabilidade física

(GO/NO-GO) com velocidade fixa. Resultado: o robô subiu +0,67 m, com inclinação máxima de 28,1°, sem capotar e com patinagem de aproximadamente 8% — veredito GO.

O treinamento PPO convergiu rapidamente (cerca de 20 episódios), atingindo recompensa média de +984,9 e 100% de sucesso — indício de que a geometria de seis rodas é mais fácil de controlar.

3.3. Otimização de Tração (Slip Ratio)

A otimização de tração consiste em identificar a aceleração na qual o robô melhor aproveita cada giro da roda, minimizando a patinagem. A métrica é o slip ratio:

$$\text{slip} = 1 - v_{\text{robo}} / (\omega \cdot \text{raio})$$

slip $\approx$ 0 → tração perfeita (todo giro vira avanço)
slip → 1 → roda patinando no vazio

Realizou-se uma varredura de throttle no robô de seis rodas (raio de roda medido da cena = 0,065 m):

throttle	v roda (m/s)	v robô (m/s)	slip (%)	ganho (m)	tilt
0,6	0,273	0,250	8,3	+0,686	18,9°
0,7	0,318	0,286	10,0	+1,035	23,3°
0,8	0,363	0,329	9,4	+1,136	23,2°
0,9	0,409	0,369	9,8	+1,336	23,4°
1,0	0,454	0,414	8,8	+1,443	23,2°

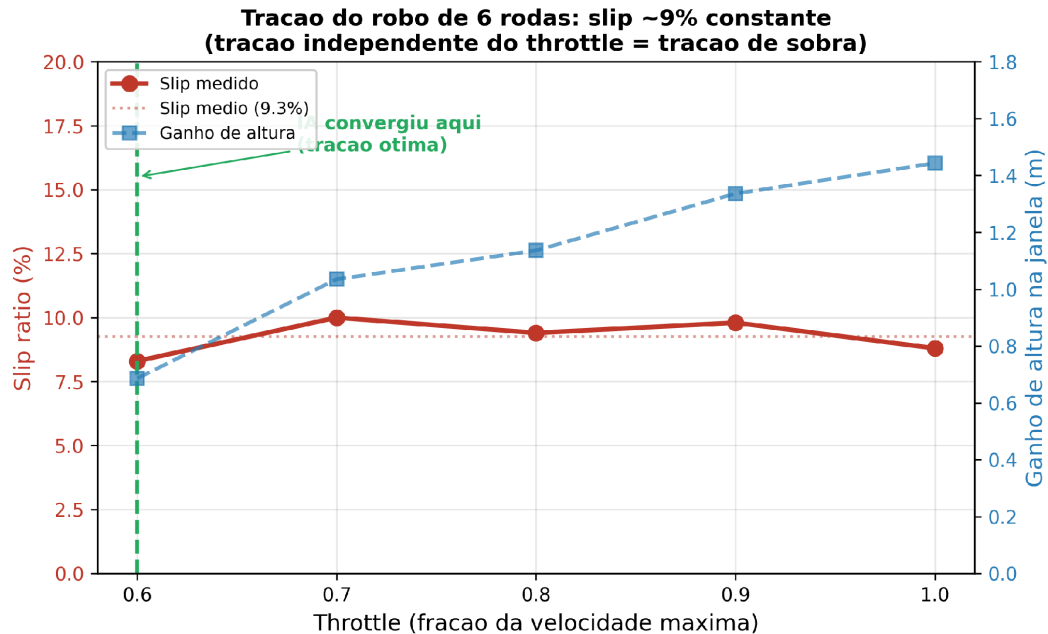


Figura 1 — Slip ratio $\times$ throttle no robô de seis rodas. A curva mantém-se praticamente constante (~9%), indicando tração de sobra; a política de RL convergiu para o throttle $\approx$ 0,6, ponto de menor slip e menor consumo.

A leitura central é que o slip permanece praticamente constante em torno de 9% em toda a faixa (as variações entre 8,3% e 10,0% são ruído de medição). O robô de seis rodas não perde tração ao acelerar — tem tração de sobra. Como a tração não é o fator limitante, a variável a

otimizar passa a ser a energia; e a política convergiu, de forma autônoma, para o throttle $\approx 0,6$, simultaneamente o ponto de menor slip e de menor consumo ($E \propto \omega^2$). Em outras palavras, a rede aprendeu o regime de tração ótima.

3.4. Comparativo Esteira × Seis Rodas

Ambas as arquiteturas foram avaliadas na mesma cena e nas mesmas três rampas, executando a missão de vencer as rampas e realizar a medição em cada patamar. Os resultados absolutos disponíveis referem-se ao robô de seis rodas; para a esteira, dispõe-se da métrica relativa de economia.

Métrica	Esteira (8 motores)	Seis Rodas (6 motores)
Taxa de sucesso	100%	100% (3/3)
Capotamento	0%	0%
Inclinação máxima	$\approx 28^\circ$	$\approx 28^\circ$
Energia — política de IA	—	41,75 J
Energia — baseline (throttle 0,8)	—	50,53 J
Economia vs baseline	$\approx 30\%$	$\approx 17,4\%$
Eficiência (J/m): IA vs baseline	—	1,392 vs 1,679
Estratégia aprendida	modula o throttle	throttle mínimo ($\approx 0,6$)

Interpretação: a porcentagem de economia mede quanto havia a ganhar, e não qual robô é superior em termos absolutos. A esteira é menos eficiente de fábrica, deixando maior margem para a IA otimizar ($\approx 30\%$). O robô de seis rodas já é eficiente e robusto por construção — restando pouca margem — de modo que a política adota o throttle mínimo e ainda assim cumpre 100% da missão ($\approx 17,4\%$). Note-se que a esteira, por concentrar a maior margem de ganho energético, é a arquitetura na qual a otimização por RL tem maior valor — e é justamente ela que foi materializada no protótipo físico.

4. Protótipo Físico: Arquitetura de Hardware

4.1. Visão Geral e Custo do Protótipo

O protótipo físico é um veículo de esteira (tracked vehicle) com tração diferencial acionada por dois motores — um por lado — comandados por uma ponte H L298N a partir de um microcontrolador ESP32. A escolha da esteira alinha o protótipo à arquitetura caterpillar da simulação, que é a arquitetura de referência do projeto.

O custo foi contido pelo reaproveitamento de componentes já disponíveis. As despesas efetivas do projeto são:

Item	Custo (R\$)
Chassi	75,00
Baterias extras	30,00
2 motores	40,00
Sensor de umidade (HW-390)	15,00
Cabos e conectores	15,00
ESP32 (reaproveitada; contabilizada como despesa do projeto)	48,40

TOTAL	≈ 223,40
-------	----------

O custo total — aproximadamente R\$ 223,40 — confirma a viabilidade econômica da arquitetura. Além dos itens adquiridos, o robô integra a IMU MPU-6050, a ponte H L298N e um micro servo, reaproveitados pelo grupo.

4.2. Pinagem e Conexões

A tabela a seguir mapeia as conexões entre o ESP32 e os periféricos, conforme definido no firmware publicado no repositório do projeto:

Periférico	Função	GPIO ESP32	Observações
L298N — Motor A (esquerdo)	IN1 (frente) / IN2 (ré)	27 / 26	Direção do motor esquerdo
L298N — Motor B (direito)	IN3 (frente) / IN4 (ré)	25 / 33	Direção do motor direito
L298N — ENA / ENB	Habilitação (velocidade)	fixo em 5 V	Sem PWM: velocidade sempre máxima
MPU-6050 (IMU)	I ² C — SDA / SCL	21 / 22	±250 °/s e ±2 g
HW-390 (umidade)	Entrada analógica (ADC)	32	Leitura de 12 bits (0–4095)
Micro servo	Sinal PWM (50 Hz)	14	Posiciona o sensor de medição
Alimentação	Motores / ESP32	—	7,4–12 V (motores); 5 V (ESP32)

Observação técnica relevante: os pinos de habilitação ENA e ENB da ponte H estão fisicamente fixados em 5 V. Isso significa que o protótipo não dispõe de modulação por largura de pulso (PWM) nos motores — a velocidade é sempre máxima, e o controle limita-se à direção (frente, ré ou parado). Essa característica tem consequências diretas sobre a validação da otimização energética, discutidas na Seção 7.

4.3. Cronograma de Desenvolvimento e Validação

O desenvolvimento físico foi organizado em cinco semanas, seguindo a lógica natural de construção: projetar, montar (Existe), instrumentar os sensores (Sente), ajustar e validar (Ajustes) e, por fim, pôr à prova (Prova).

Semana	Foco	Entregável
1 — Projeto	Projeto mecânico e elétrico; BOM; compra das peças	Projeto + pedido de compras
2 — Existe	Montagem mecânica do chassi de esteira	Chassi rodando livre
3 — Sente	Eletrônica energizada; integração dos sensores (IMU e umidade)	Sensores lendo
4 — Ajustes	Ajustes necessários e validação dos sensores	Sensores validados
5 — Prova	Ensaio de mobilidade em rampa; consolidação dos resultados	Documentação final

5. Firmware Embarcado (ESP32)

O firmware foi desenvolvido em C++ no ecossistema Arduino para o ESP32 e está publicado em <https://github.com/MF853/Calango-Tech>. As bibliotecas utilizadas são: Bluepad32 (Ricardo Quesada) para o pareamento do controle Xbox, MPU6050 (jrowberg) para a IMU, ESP32Servo para o servo de medição e WiFi/WebServer (nativas) para a telemetria em rede.

5.1. Controle Tank Drive via Xbox One S

O robô é teleoperado por um controle Xbox One S pareado por Bluetooth Classic (BR/EDR) — e não BLE, distinção crítica que motiva o uso da biblioteca Bluepad32. O esquema de condução é o tank drive: cada analógico comanda independentemente o motor do seu lado, permitindo avanço, ré e giro sobre o próprio eixo (pivô). O mapeamento de comandos é:

Comando	Ação
Analógico esquerdo	Aciona o motor esquerdo (tank drive)
Analógico direito	Aciona o motor direito (tank drive)
Gatilhos RT / LT	Aceleração / ré — sobrepõem o tank drive
Botão A	Dispara a captura de dados (inclinação + umidade)
Botão B	Parada de emergência

Para evitar acionamento indevido por deriva dos analógicos em repouso, o firmware aplica zonas mortas (deadzones). A função de acionamento recebe o valor bruto do analógico (−512 a +511): valores dentro da zona morta param o motor; valores positivos ou negativos acionam, respectivamente, os pinos de frente ou de ré. As constantes de calibração são:

```
#define DEADZONE          40    // zona morta dos analogicos
#define TRIGGER_DEADZONE  30    // zona morta dos gatilhos RT/LT
#define ALPHA              0.98 // peso do filtro complementar (IMU)
#define HW390_SECO        3200  // leitura ADC em solo seco
#define HW390_MOLHADO     800   // leitura ADC em solo molhado
#define SERVO_REPOUSO     0     // graus - posicao recolhida
#define SERVO_MEDICAO     90    // graus - posicao de medicao
```

5.2. Leitura da IMU e Filtro Complementar

A MPU-6050 é inicializada com fundo de escala de ± 250 °/s para o giroscópio e ± 2 g para o acelerômetro — faixas adequadas a um veículo terrestre lento, maximizando a resolução. A estimativa de atitude combina as duas fontes por um filtro complementar com peso ALPHA = 0,98, que integra a velocidade angular do giroscópio no curto prazo e corrige a deriva com a referência gravitacional do acelerômetro no longo prazo, produzindo os ângulos de pitch (frente/trás) e roll (lateral).

Esta é a contrapartida física da detecção de inclinação da simulação (Seção 2.4): ali, a inclinação vinha da matriz de rotação do simulador; aqui, é reconstruída a partir de sensores inerciais reais e do filtro complementar.

5.3. Medição de Umidade com Servo

A medição da grandeza ambiental é realizada por um sensor capacitivo de umidade de solo HW-390, lido pelo conversor analógico-digital de 12 bits do ESP32 (0 a 4095). A conversão para porcentagem utiliza dois pontos de calibração — solo seco (3200) e solo molhado (800)

— mapeados para a faixa de 0 a 100%. Note-se que a relação é inversa: quanto maior a umidade, menor a leitura bruta do ADC.

O diferencial da implementação é o acionamento por servo. A rotina de captura, disparada pelo botão A do controle, executa a seguinte sequência: registra os ângulos atuais da IMU, gira o servo da posição de repouso (0°) para a posição de medição (90°), coleta a média de dez amostras do ADC ao longo de 50 ms — suavizando o ruído da leitura analógica —, registra os resultados no monitor serial e recolhe o servo. Trata-se, portanto, de uma medição sob demanda, com o sensor protegido durante o deslocamento.

5.4. Telemetria: Monitor Serial e Dashboard Web

A telemetria possui duas vias. A primeira é o monitor serial, no qual cada captura é impressa de forma legível (Seção 6.1). A segunda, presente na versão em rede do firmware, é um servidor HTTP embarcado no próprio ESP32, que expõe dois recursos:

- GET / — serve um painel (dashboard) de tema escuro com indicadores gráficos em SVG, atualizados em tempo real.
- GET /dados — retorna um objeto JSON contendo pitch, roll, leitura bruta do ADC, umidade em porcentagem, posição do servo, estado do gamepad e o histórico de leituras arquivadas.

A separação entre a interface (dashboard) e o contrato de dados (JSON) permite consumir a telemetria tanto por um navegador quanto por scripts de análise, sem alterar o firmware.

5.5. Versões do Firmware

O repositório disponibiliza duas versões do sketch, com a mesma base de controle e percepção, diferindo apenas na conectividade:

Sketch	Conectividade	Recursos
carrinho_xbox	Bluetooth Classic	Tank drive, MPU-6050, HW-390, servo e telemetria serial
carrinho_xbox_server	Bluetooth Classic + WiFi	Todos os recursos acima, acrescidos do dashboard web e do endpoint JSON /dados

6. Resultados do Protótipo Físico

6.1. Validação dos Sensores

Os dois sensores de percepção foram validados com sucesso. A leitura do monitor serial do ESP32 (Figura 2) confirma o funcionamento do conjunto giroscópio/acelerômetro MPU-6050 (pitch = $-5,56^\circ$ e roll = $-13,07^\circ$) e do sensor de umidade capacitivo HW-390 (leitura ADC = 3108), ambos acionados pela rotina de captura do botão A.

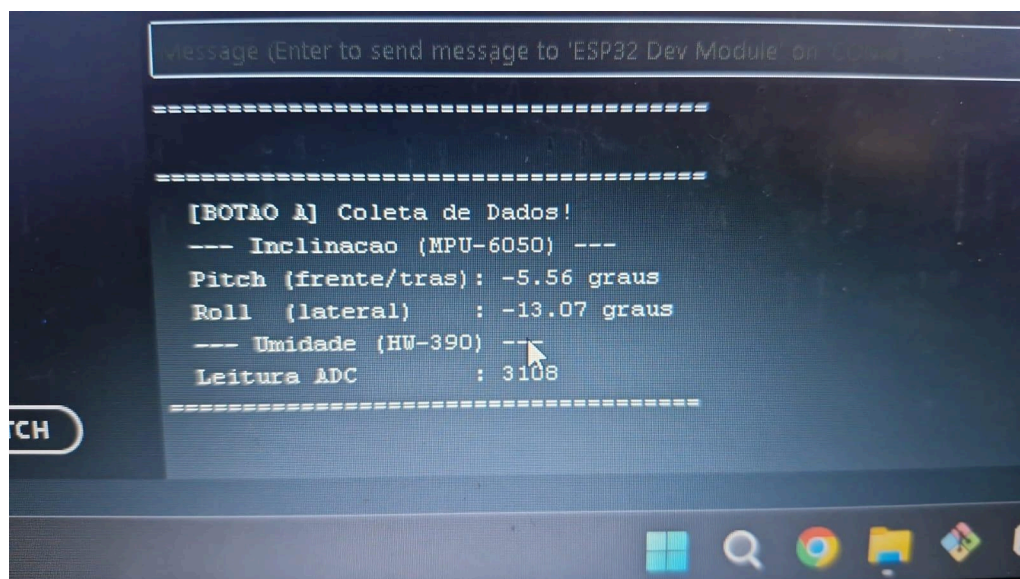


Figura 2 — Monitor serial do ESP32 exibindo a coleta de dados: inclinação (MPU-6050) e umidade (HW-390) funcionando corretamente.

6.2. Testes de Mobilidade em Rampa

Os ensaios de mobilidade foram conduzidos em rampa rígida (superfície plana, tipo tábua), com registro em vídeo nos ângulos de 15°, 30°, 35° e 45°, além da série de ensaios finais (TesteReal1 a TesteReal6). O robô apresentou mobilidade consistente em inclinações de até cerca de 25° a 27°. A partir de aproximadamente 30°, ocorre empacamento intermitente, com o robô por vezes não conseguindo vencer a subida.

A causa do empacamento é a combinação de dois fatores: a limitação de fornecimento de corrente da bateria para acionar os dois motores simultaneamente e a ausência de modulação PWM, que impede dosar a força aplicada (Seção 4.2).

Quanto à natureza da evidência, cabe um registro metodológico. Os ensaios realizados têm caráter exploratório: constituem um conjunto reduzido de execuções, sem repetições sistemáticas por condição (ângulo da rampa, carga da bateria e ponto de partida). Por essa razão, não é possível derivar deles taxas de sucesso estatisticamente significativas, ao contrário do que ocorre na frente de simulação, onde o volume de episódios sustenta os percentuais reportados. Os resultados físicos aqui apresentados devem, portanto, ser lidos como indicativos de comportamento — evidência consistente, porém preliminar — e não como métricas consolidadas. A ampliação da campanha de ensaios para uma escala maior é condição necessária para elevá-los ao mesmo rigor estatístico da simulação (Seção 9).

6.3. Limitações Identificadas

De forma transparente, nem todos os indicadores previstos para o protótipo físico foram atingidos. As limitações identificadas são:

- Ausência de modulação PWM: com ENA/ENB fixos em 5 V, o robô opera sempre em velocidade máxima, restando ao operador apenas o comando de direção. Isso reduz a dosagem de força disponível em rampas íngremes e contribui para o empacamento.

- Alimentação insuficiente: a bateria atual não fornece corrente suficiente para acionar os dois motores simultaneamente com a força ideal, provocando empacamento em rampas mais íngremes.
- Terreno solto não testado: os ensaios ocorreram apenas em superfície rígida. O comportamento em barro, terra ou areia — mais próximo da aplicação real — ainda não foi avaliado. É a principal pendência experimental.
- Escala de ensaios reduzida: os testes realizados compõem um conjunto pequeno de execuções, sem repetições sistemáticas por condição. A evidência é consistente, mas preliminar, e precisa ser ampliada para sustentar conclusões estatísticas comparáveis às da simulação.
- Energia e patinagem não instrumentadas: o protótipo não possui sensor de corrente nem encoders, de modo que a energia real e o slip real não puderam ser medidos.
- Em contrapartida, a percepção está plenamente funcional: IMU (MPU-6050) e sensor de umidade (HW-390) validados, com medição sob demanda por servo e telemetria por serial e dashboard web.

7. Comparativo Simulação × Realidade (Gap Sim-to-Real)

Esta seção confronta o que foi alcançado em cada frente, distinguindo com clareza o que decorre de decisão de escopo e o que constitui pendência experimental. Registre-se, de início, que as duas frentes têm objetivos distintos por projeto: a simulação investiga a otimização do controle, enquanto o protótipo valida mecânica, percepção e mobilidade sob teleoperação.

Aspecto	Simulação	Protótipo Físico	Situação
Arquitetura	esteira e seis rodas	esteira	Alinhado
Locomoção	política PPO (escopo da simulação)	controle Bluetooth (escopo do protótipo)	Conforme escopo
Inclinação	tilt por matriz de rotação	MPU-6050 + filtro complementar	Alinhado
Umidade	proxy simulado	HW-390 + servo de medição	Alinhado
Modulação de velocidade	throttle contínuo [0,6; 1,0]	sem PWM (100% fixo)	Divergente
Energia	proxy $E = \Sigma(0,005 \cdot \omega^2)$	não instrumentada	Pendente
Patinagem (slip)	medida na varredura	não instrumentada	Pendente
Subida em rampa rígida	até $\approx 28^\circ$ (100%)	$\approx 25\text{--}27^\circ$ (parcial)	Parcial
Terreno solto	não aplicável	não testado	Pendente

Na frente de percepção, o alinhamento é integral: os dois sentidos que a missão exige — inclinação e umidade — foram materializados e validados no hardware real, com um mecanismo de medição por servo que sequer existia na simulação. A arquitetura de esteira, por sua vez, é comum às duas frentes.

A principal divergência técnica está na modulação de velocidade: como os pinos ENA/ENB estão fixos em 5 V, o protótipo não dispõe de PWM e opera sempre em velocidade máxima. Trata-se de uma característica da eletrônica de acionamento adotada, e não de uma falha de projeto — mas é o fator que, somado à capacidade de corrente da bateria, explica o

empacamento em rampas acima de aproximadamente 30°. Habilitar o PWM seria, também, o pré-requisito caso se deseje futuramente transpor ao hardware a otimização energética estudada em simulação.

8. Guia Passo a Passo de Execução

Esta seção reúne os procedimentos necessários para reproduzir tanto a frente de simulação quanto a operação do protótipo físico.

8.1. Pré-requisitos

- Simulação: CoppeliaSim 4.10 e Python, com as bibliotecas `coppelasim_zmqremoteapi_client`, `stable-baselines3`, `gymnasium`, `numpy` e `matplotlib`.
- Firmware: Arduino IDE com o suporte a placas ESP32 (Espressif) e as bibliotecas `Bluepad32` (Ricardo Quesada), `MPU6050` (jrowberg) e `ESP32Servo`.

8.2. Executando a Simulação

Antes de rodar qualquer script, abra a cena no CoppeliaSim e posicione o robô no pé da primeira rampa, voltado para a subida. Não inicie a simulação manualmente — os scripts assumem o controle do simulador via ZMQ Remote API.

```
python treinar_robo.py           # treina a politica PPO (esteira)
python testar_robo.py           # executa a missao com a IA treinada
python testar_robo.py --constante # executa o baseline de throttle fixo

python teste_6rodas_gonogo.py    # viabilidade fisica do robo de 6 rodas
python treinar_robo_6rodas.py    # treina a politica PPO (6 rodas)
python testar_robo_6rodas.py     # missao do robo de 6 rodas

python medir_tracao.py          # varredura de slip x throttle
python grafico_tracao.py        # gera o grafico slip x throttle
```

O acompanhamento do treinamento pode ser feito por TensorBoard. Os scripts de missão imprimem, ao final, a energia total consumida — é a comparação entre a execução com a IA e a execução com `--constante` que produz a métrica de economia energética.

8.3. Gravando e Operando o Protótipo Físico

- No Arduino IDE, selecione a placa ESP32 Dev Module e configure a velocidade de upload em 115200 bps.
- Abra o sketch desejado — `carrinho_xbox.ino` (apenas Bluetooth) ou `carrinho_xbox_server.ino` (Bluetooth e WiFi com dashboard) — e grave-o na placa.
- Ligue a alimentação dos motores (7,4–12 V) e coloque o controle Xbox One S em modo de pareamento, pressionando o seu botão dedicado de pareamento. O ESP32 conecta-se automaticamente por Bluetooth Classic.
- Conduza o robô pelos analógicos (tank drive) ou pelos gatilhos RT/LT. Pressione o botão A para capturar inclinação e umidade, e o botão B para a parada de emergência.
- Na versão `carrinho_xbox_server`, acesse pelo navegador o endereço IP atribuído ao ESP32 para visualizar o dashboard; o recurso `/dados` devolve as leituras em JSON.

Solução de problemas comuns: falha de pareamento normalmente exige o botão dedicado do controle; inversão de sentido dos motores resolve-se trocando fisicamente os fios de saída da ponte H; trepidação em baixa velocidade é ajustada pelas constantes de zona morta; e instabilidade de alimentação é mitigada pela adição de um capacitor.

9. Conclusões e Trabalhos Futuros

O Projeto Calango-Tech consolidou uma frente de simulação completa e validada e um protótipo físico funcional, com escopos honestamente delimitados. Os resultados consolidados são:

- Simulação — esteira: 100% de sucesso, 0% de capotamento, pitch máximo $\approx 28^\circ$ e $\approx 30\%$ de economia de energia frente ao baseline, com dominância de Pareto em energia $\times$ tempo.
- Simulação — seis rodas: 100% de sucesso e $\approx 17,4\%$ de economia (41,75 J contra 50,53 J), com a política convergindo autonomamente para o throttle de tração ótima ($\approx 0,6$).
- Otimização de tração: slip praticamente constante ($\sim 9\%$) em toda a faixa de throttle, evidenciando tração de sobra; a política aprendeu, sozinha, o regime de menor patinagem e menor consumo.
- Metodologia: o action shaping reduziu a convergência do treinamento de ~ 10 para 2 iterações, e a detecção de capotamento por matriz de rotação eliminou a ambiguidade dos ângulos de Euler.
- Protótipo físico: veículo de esteira teleoperado por Xbox via Bluetooth, custo de $\approx$ R\$ 223,40, com IMU e sensor de umidade validados, medição sob demanda por servo, telemetria serial e dashboard web embarcado.

Os trabalhos futuros, em ordem de prioridade, são:

- Substituir a alimentação por uma fonte capaz de suprir os dois motores simultaneamente, eliminando o empacamento em rampas íngremes.
- Implementar PWM nos pinos ENA/ENB da ponte H, permitindo ao operador dosar a velocidade e melhorar a transposição de rampas.
- Ampliar a campanha de ensaios para uma escala maior, com repetições sistemáticas por ângulo de rampa e por condição de bateria, de modo a converter a evidência preliminar atual em taxas de sucesso e métricas físicas estatisticamente comparáveis às da simulação.
- Ensaiar o robô em terreno solto (barro, terra e areia), aproximando os testes da aplicação real.
- Instrumentar a medição de energia real (sensor de corrente) e de patinagem real (encoders nos motores), tornando comparáveis as métricas de simulação e realidade.